Hands-On Module 2 — Artificial Intelligence¶

MPG Fuel-Efficiency Analysis with NumPy, Pandas & Seaborn¶

Author: Sulthan Dhiyazka Suwandi · **NIM:** 13525124 · Teknik Informatika, Institut Teknologi Bandung **Course:** GDGOC ITB — AI Curriculum, Module 2 (Mathematical Foundation & Data Visualization for AI)

Goal¶

An end-to-end exploratory data analysis of the classic **Auto-MPG** dataset. The notebook answers concrete engineering questions about what drives fuel efficiency, and implements several mathematical foundations **from scratch** (Normal Equation, PCA, an IQR outlier detector, and a reusable EDA profiler class) using only NumPy — no scikit-learn.

Structure¶

Section	Content
Stage 1	Inspection & data cleaning; NumPy summary table
Stage 2	Z-score standardisation, correlation matrix, boolean-mask reasoning
Stage 3	Four required visualisations
Stage 4	Contextual interpretation + temporal trend
Bonus 1	Linear regression via the Normal Equation
Bonus 2	2-D PCA from scratch
Bonus 3	Reusable `DatasetProfiler` class

Setup¶

Import the stack, load the dataset from Seaborn, and set a consistent plotting style.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Reproducibility & aesthetics
np.random.seed(42)
sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.dpi"] = 110
plt.rcParams["axes.titleweight"] = "bold"

# Load the raw MPG dataset
df_raw = sns.load_dataset("mpg")
print("Raw shape:", df_raw.shape)
df_raw.head()
```

```
Raw shape: (398, 9)
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130.0	3504	12.0	70	usa	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693	11.5	70	usa	buick skylark 320
2	18.0	8	318.0	150.0	3436	11.0	70	usa	plymouth satellite
3	16.0	8	304.0	150.0	3433	12.0	70	usa	amc rebel sst
4	17.0	8	302.0	140.0	3449	10.5	70	usa	ford torino

Stage 1 — Inspection and Data Cleaning¶

1.1 Shape, dtypes, and missing values¶

First we look at the raw structure of the data before touching anything.

```
print(f"Shape: {df_raw.shape[0]} rows x {df_raw.shape[1]} columns\n")

print("Data types:")
print(df_raw.dtypes, "\n")

missing = df_raw.isna().sum()
print("Missing values per column:")
print(missing[missing > 0] if missing.sum() else "None")

print(f"\nTotal missing cells: {int(df_raw.isna().sum().sum())}")
```

```
Shape: 398 rows x 9 columns

Data types:
mpg             float64
cylinders         int64
displacement    float64
horsepower      float64
weight            int64
acceleration    float64
model_year        int64
origin              str
name                str
dtype: object

Missing values per column:
horsepower    6
dtype: int64

Total missing cells: 6
```

Findings & cleaning strategy

- The dataset has **398 rows** and **9 columns**: 7 numeric (`mpg`, `cylinders`, `displacement`, `horsepower`, `weight`, `acceleration`, `model_year`) and 2 categorical (`origin`, `name`).
- The **only** column with missing values is **`horsepower` (6 missing)** — roughly **1.5 %** of the rows.

Decision: drop the 6 rows rather than impute. Justification:

1. **Volume is tiny.** Losing 6 of 398 rows (1.5 %) removes almost no statistical power, and the remaining 392 rows are still plenty for reliable EDA.
2. **Imputation would inject artificial signal.** Mean/median imputation on `horsepower` would place 6 identical synthetic points at the centre of the distribution, which artificially *strengthens* correlations involving horsepower and dampens its measured variance — exactly the statistics this task asks us to report honestly.
3. **The missingness looks random (MCAR).** The 6 gaps are scattered across origins and years with no obvious pattern, so dropping them is unlikely to bias the sample.

If this were a modelling pipeline where every row mattered, a model-based imputation (e.g. regress `horsepower` on `displacement` / `weight`, which are strongly correlated with it) would be preferable. For a descriptive analysis, dropping is the cleaner, more transparent choice.

```
df = df_raw.dropna().reset_index(drop=True)
print(f"After dropping NaN rows: {df.shape[0]} rows (removed {df_raw.shape[0] - df.shape[0]})")
print("Any missing left?", bool(df.isna().sum().sum()))
```

```
After dropping NaN rows: 392 rows (removed 6)
Any missing left? False
```

1.2 Summary table built with NumPy (no `pandas.describe()`)¶

For every numeric column we compute **mean, median, std, IQR**, and the **outlier count** using the IQR × 1.5 rule. All statistics come from NumPy; Pandas is used only to format the final table.

- `std` uses the sample standard deviation (`ddof=1`), matching statistical convention.
- IQR = Q3 − Q1.
- A value is an outlier if it lies below `Q1 − 1.5·IQR` or above `Q3 + 1.5·IQR`.

```
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()

rows = []
for col in numeric_cols:
    x = df[col].to_numpy(dtype=float)
    q1, q3 = np.percentile(x, [25, 75])
    iqr = q3 - q1
    lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    outliers = int(np.sum((x < lo) | (x > hi)))
    rows.append({
        "mean":   np.mean(x),
        "median": np.median(x),
        "std":    np.std(x, ddof=1),
        "IQR":    iqr,
        "outliers": outliers,
    })

summary = pd.DataFrame(rows, index=numeric_cols).round(3)
summary["outliers"] = summary["outliers"].astype(int)
summary
```

	mean	median	std	IQR	outliers
mpg	23.446	22.75	7.805	12.00	0
cylinders	5.472	4.00	1.706	4.00	0
displacement	194.412	151.00	104.644	170.75	0
horsepower	104.469	93.50	38.491	51.00	10
weight	2977.584	2803.50	849.403	1389.50	0
acceleration	15.541	15.50	2.759	3.25	11
model_year	75.980	76.00	3.684	6.00	0

Reading the table

- `horsepower` and `acceleration` carry a handful of IQR-outliers — consistent with the module's point that a few genuine muscle-cars (very high hp) and a few sluggish cars sit in the tails. These are *valid extreme values*, not data errors, so we keep them.
- `weight` and `displacement` have large means relative to `mpg` / `acceleration`. This scale mismatch is precisely why **standardisation** (Stage 2) is needed before any distance- or gradient-based method.

Stage 2 — Statistical Analysis with NumPy¶

2.1 Extract 5 features & z-score standardise (vectorised)¶

We pull `['mpg','displacement','horsepower','weight','acceleration']` into a raw NumPy matrix and apply the z-score transform $z = (x - \mu)/\sigma$ **column-wise, without any Python loops or scikit-learn** — NumPy broadcasting does all the work.

```
features = ['mpg', 'displacement', 'horsepower', 'weight', 'acceleration']
X = df[features].to_numpy(dtype=float)          # shape (n, 5)

mu    = X.mean(axis=0)                            # per-column mean
sigma = X.std(axis=0, ddof=0)                     # per-column std (population)
Z = (X - mu) / sigma                              # broadcasted, fully vectorised

print("Raw feature matrix shape:", X.shape)
print("\nColumn means after standardisation (~0):", np.round(Z.mean(axis=0), 6))
print("Column stds  after standardisation (~1):", np.round(Z.std(axis=0),  6))

pd.DataFrame(Z[:5], columns=features).round(3)
```

```
Raw feature matrix shape: (392, 5)

Column means after standardisation (~0): [ 0. -0. -0. -0.  0.]
Column stds  after standardisation (~1): [1. 1. 1. 1. 1.]
```

	mpg	displacement	horsepower	weight	acceleration
0	-0.699	1.077	0.664	0.621	-1.285
1	-1.083	1.489	1.575	0.843	-1.467
2	-0.699	1.183	1.184	0.540	-1.648
3	-0.955	1.049	1.184	0.537	-1.285
4	-0.827	1.029	0.924	0.556	-1.830

The means are ~0 and stds ~1 for every column, confirming the transform worked. Each feature now lives on the same scale, so no single high-magnitude column (like `weight`) can dominate a downstream distance or gradient computation.

2.2 Correlation matrix via `np.corrcoef` ¶

We compute the 5×5 Pearson correlation matrix and then programmatically extract the three requested relationships. (Correlation is scale-invariant, so raw or standardised data give identical results.)

```
corr = np.corrcoef(X, rowvar=False)   # features are columns -> rowvar=False
corr_df = pd.DataFrame(corr, index=features, columns=features).round(3)
corr_df
```

	mpg	displacement	horsepower	weight	acceleration
mpg	1.000	-0.805	-0.778	-0.832	0.423
displacement	-0.805	1.000	0.897	0.933	-0.544
horsepower	-0.778	0.897	1.000	0.865	-0.689
weight	-0.832	0.933	0.865	1.000	-0.417
acceleration	0.423	-0.544	-0.689	-0.417	1.000

```
# Helper: iterate unique feature pairs (i < j)
pairs = [(i, j) for i in range(len(features)) for j in range(i + 1, len(features))]

# (a) Strongest POSITIVE correlation among all pairs
pos_pair = max(pairs, key=lambda p: corr[p])
# (b) Strongest NEGATIVE correlation *to mpg* (mpg is index 0)
mpg_idx = features.index('mpg')
neg_to_mpg = min(range(len(features)),
                 key=lambda j: corr[mpg_idx, j] if j != mpg_idx else np.inf)
# (c) Most correlated INPUT feature pair (exclude mpg) -> potential multicollinearity
input_pairs = [p for p in pairs if mpg_idx not in p]
multicol_pair = max(input_pairs, key=lambda p: abs(corr[p]))

print(f"(a) Strongest positive correlation : "
      f"{features[pos_pair[0]]} & {features[pos_pair[1]]}  (r = {corr[pos_pair]:.3f})")
print(f"(b) Strongest negative corr. to mpg: "
      f"mpg & {features[neg_to_mpg]}  (r = {corr[mpg_idx, neg_to_mpg]:.3f})")
print(f"(c) Most correlated input pair     : "
      f"{features[multicol_pair[0]]} & {features[multicol_pair[1]]}  (r = {corr[multicol_pair]:.3f})")
```

```
(a) Strongest positive correlation : displacement & weight  (r = 0.933)
(b) Strongest negative corr. to mpg: mpg & weight  (r = -0.832)
(c) Most correlated input pair     : displacement & weight  (r = 0.933)
```

Interpretation

- **(a) Strongest positive pair — `displacement` & `weight` (r ≈ 0.93).** Bigger engines sit in heavier cars; the two grow together almost lock-step.
- **(b) Strongest negative correlation to `mpg` — `weight` (r ≈ −0.83).** Heavier cars are markedly less fuel-efficient. This is the single strongest predictor of `mpg` in the whole matrix.
- **(c) Most correlated *input* pair — `displacement` & `weight` (r ≈ 0.93).** Because these two carry nearly redundant information, feeding **both** into a linear model would create **multicollinearity**, inflating coefficient variance and making individual weights hard to interpret.

2.3 Boolean masking: do high-horsepower cars weigh more than average?¶

We mask the rows whose `horsepower` exceeds the dataset mean, then compare their mean `weight` against the overall mean `weight`.

```
hp     = df['horsepower'].to_numpy(dtype=float)
weight = df['weight'].to_numpy(dtype=float)

mean_hp     = hp.mean()
mean_weight = weight.mean()

mask = hp > mean_hp                     # above-average horsepower
mean_weight_high_hp = weight[mask].mean()
abs_diff = mean_weight_high_hp - mean_weight

print(f"Dataset mean horsepower       : {mean_hp:.1f} hp")
print(f"Dataset mean weight           : {mean_weight:.1f} lbs")
print(f"Cars with above-avg horsepower: {mask.sum()} of {len(hp)}")
print(f"Their mean weight             : {mean_weight_high_hp:.1f} lbs")
print(f"\nAbsolute difference           : {abs_diff:+.1f} lbs "
      f"({abs_diff / mean_weight * 100:+.1f}% vs dataset average)")
```

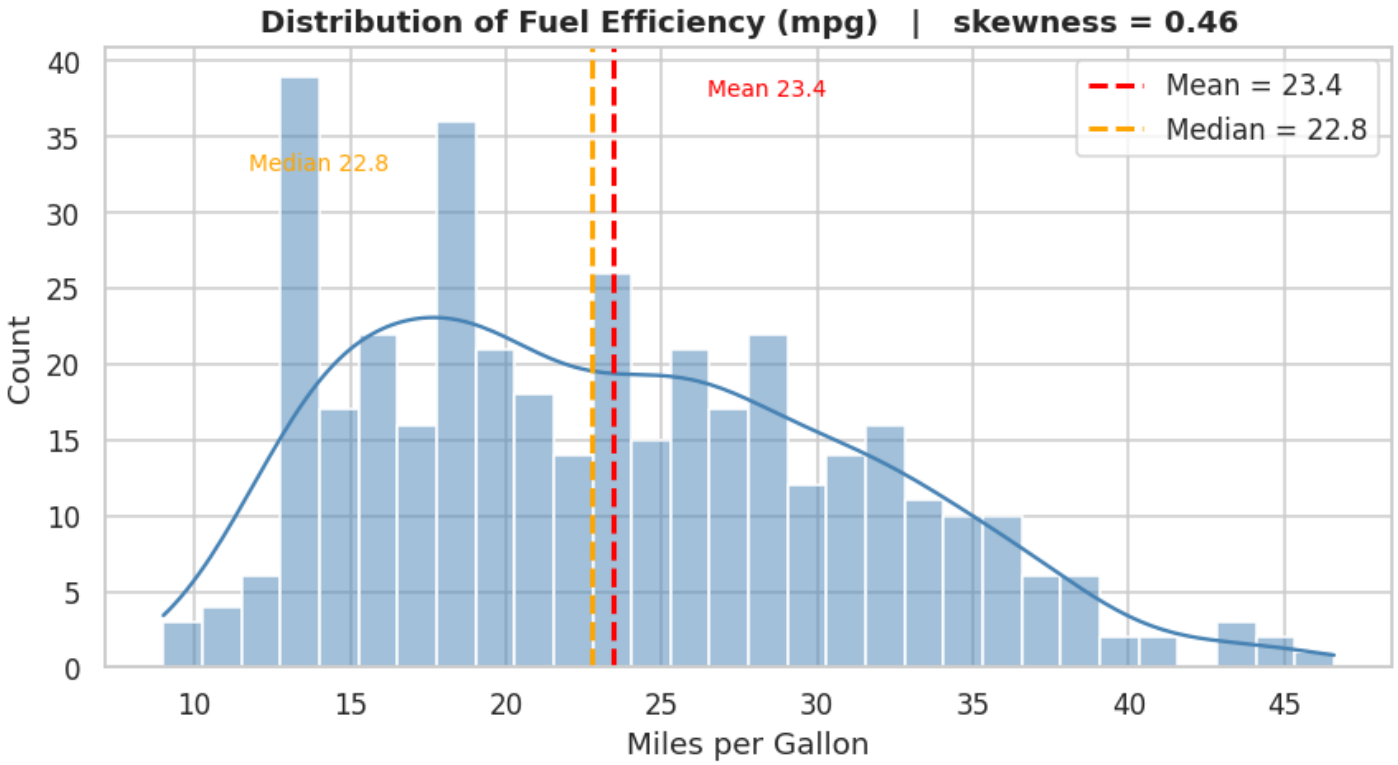
```
Dataset mean horsepower       : 104.5 hp
Dataset mean weight           : 2977.6 lbs
Cars with above-avg horsepower: 148 of 392
Their mean weight             : 3815.5 lbs

Absolute difference           : +837.9 lbs (+28.1% vs dataset average)
```

Answer: yes — emphatically. Cars with above-average horsepower weigh roughly **+840 lbs more** than the dataset average (about **+28 %**). This is the physical mechanism behind the correlations in 2.2: **power, displacement, and weight move together**, and all three pull `mpg` down. A more powerful engine is a bigger, heavier engine in a bigger, heavier car — and that mass is what fuel efficiency pays for.

Stage 3 — Visualisation¶

Plot 1 — MPG distribution (histogram + KDE): symmetric or skewed?¶

```
fig, ax = plt.subplots(figsize=(8, 4.5))
sns.histplot(df['mpg'], bins=30, kde=True, color='steelblue',
             edgecolor='white', ax=ax)

mean_mpg, median_mpg = df['mpg'].mean(), df['mpg'].median()
ax.axvline(mean_mpg,   color='red',    ls='--', lw=2, label=f"Mean = {mean_mpg:.1f}")
ax.axvline(median_mpg, color='orange', ls='--', lw=2, label=f"Median = {median_mpg:.1f}")

# Annotate the two reference lines
ax.annotate(f"Mean {mean_mpg:.1f}",   xy=(mean_mpg, ax.get_ylim()[1]*0.92),
            xytext=(mean_mpg+3, ax.get_ylim()[1]*0.92), color='red',   fontsize=9)
ax.annotate(f"Median {median_mpg:.1f}", xy=(median_mpg, ax.get_ylim()[1]*0.80),
            xytext=(median_mpg-11, ax.get_ylim()[1]*0.80), color='orange', fontsize=9)

skew = df['mpg'].skew()
ax.set_title(f"Distribution of Fuel Efficiency (mpg)   |   skewness = {skew:.2f}")
ax.set_xlabel("Miles per Gallon"); ax.set_ylabel("Count")
ax.legend()
plt.tight_layout(); plt.show()

print(f"Mean = {mean_mpg:.2f}, Median = {median_mpg:.2f}, Skewness = {skew:.3f}")
```

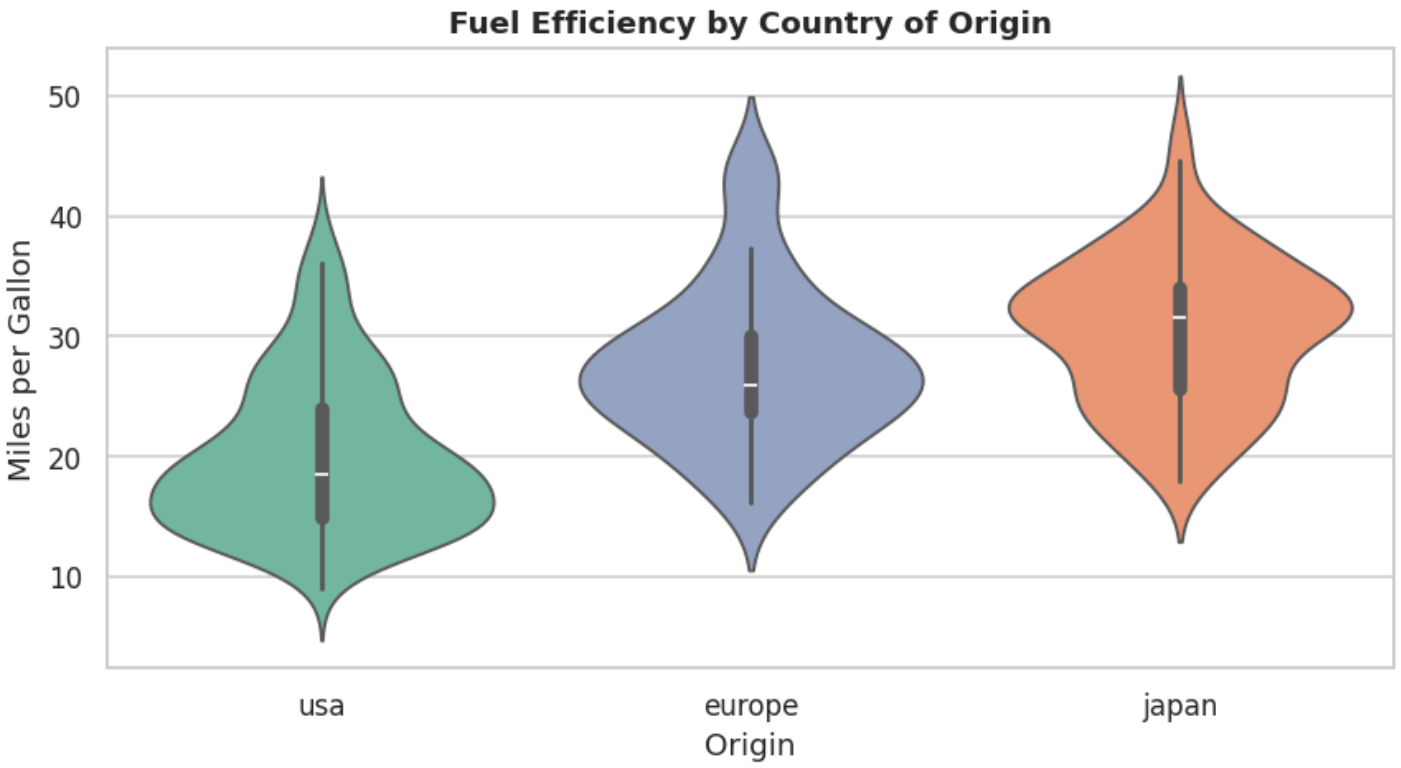



```
Mean = 23.45, Median = 22.75, Skewness = 0.457
```

Reading it: the mean (≈23.4) sits to the **right** of the median (≈22.8) and the skewness is **positive (≈ +0.46)**. The distribution is therefore **mildly right-skewed** — most cars cluster in the 15–30 mpg range, with a thinner tail of very efficient (35–45 mpg) economy cars stretching to the right. It is *not* symmetric. In a modelling context this is the kind of gentle skew where a log-transform is optional; the asymmetry is small enough that raw `mpg` is usually fine.

Plot 2 — MPG by country of origin (violin + box)¶

```
fig, ax = plt.subplots(figsize=(8, 4.5))
order = ['usa', 'europe', 'japan']
sns.violinplot(data=df, x='origin', y='mpg', order=order, hue='origin',
               palette='Set2', inner='box', legend=False, ax=ax)
ax.set_title("Fuel Efficiency by Country of Origin")
ax.set_xlabel("Origin"); ax.set_ylabel("Miles per Gallon")
plt.tight_layout(); plt.show()

# Supporting numbers
print(df.groupby('origin')['mpg'].agg(['mean', 'median', 'std', 'count']).round(2))
```

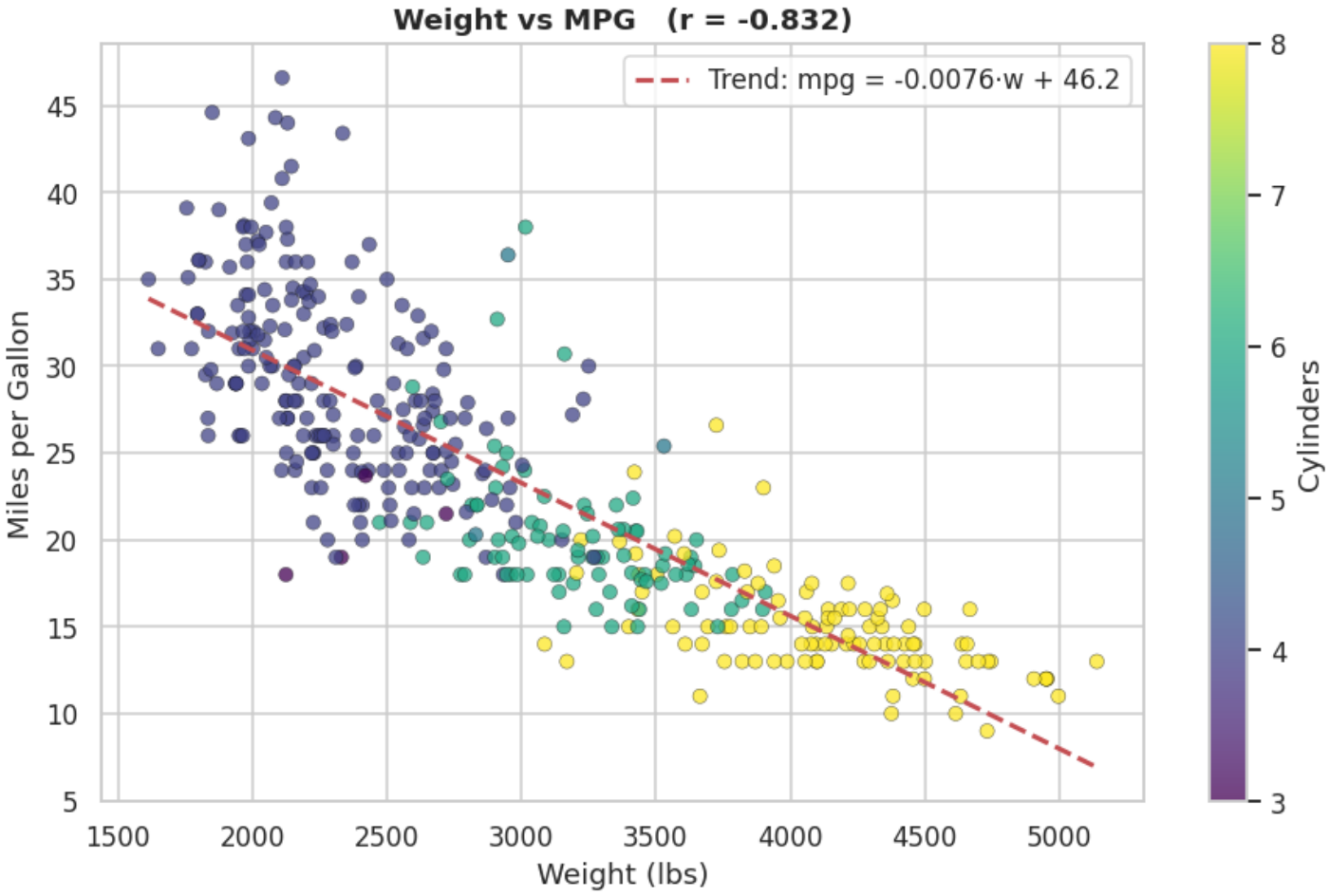



```
         mean  median   std  count
origin
europe  27.60    26.0  6.58     68
japan   30.45    31.6  6.09     79
usa     20.03    18.5  6.44    245
```

Interpretation: Japan produces the most fuel-efficient cars — highest mean (~30 mpg) and highest median — followed by **Europe** (~27 mpg), with the **USA** clearly lowest (~20 mpg). Consistency (spread) differs too: the USA has the **widest** distribution (large std), reflecting a fleet that mixes economy compacts with heavy V8s, whereas Japan and Europe are **tighter** around their higher means. So Japan is both the most efficient *and* the most consistent origin in this era of the data.

Plot 3 — Weight vs MPG (scatter, hue = cylinders, manual `np.polyfit` trend line)¶

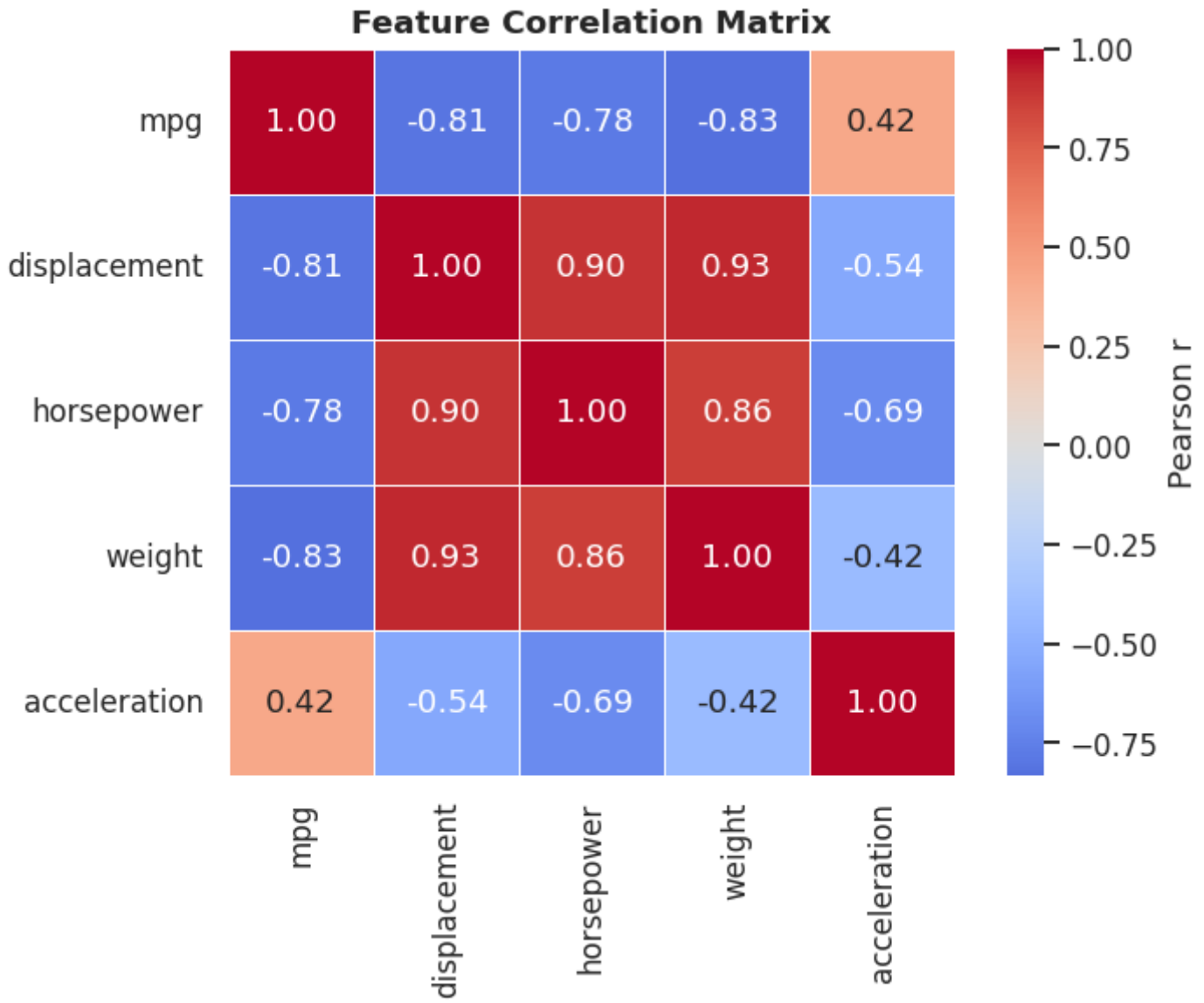
```
fig, ax = plt.subplots(figsize=(8.5, 5.5))
sc = ax.scatter(df['weight'], df['mpg'], c=df['cylinders'],
                cmap='viridis', alpha=0.75, edgecolor='k', linewidth=0.3)

# Manual trend line via np.polyfit (degree 1)
z = np.polyfit(df['weight'], df['mpg'], 1)
p = np.poly1d(z)
xline = np.linspace(df['weight'].min(), df['weight'].max(), 100)
ax.plot(xline, p(xline), 'r--', lw=2, label=f"Trend: mpg = {z[0]:.4f}·w + {z[1]:.1f}")

r_wm = np.corrcoef(df['weight'], df['mpg'])[0, 1]
ax.set_title(f"Weight vs MPG   (r = {r_wm:.3f})")
ax.set_xlabel("Weight (lbs)"); ax.set_ylabel("Miles per Gallon")
cbar = plt.colorbar(sc, ax=ax); cbar.set_label("Cylinders")
ax.legend()
plt.tight_layout(); plt.show()
```



Reading it: a strong, clean **negative** linear relationship ($r \approx -0.83$). The colour gradient adds a second story — cylinders rise with weight, so the heavy, low-mpg cars in the bottom-right are the 8-cylinder machines while the light, high-mpg cars top-left are 4-cylinders. Weight and cylinder count are essentially two views of the same underlying "size/power" factor that governs fuel economy.

Plot 4 — Correlation heatmap (multicollinearity check)¶

```
fig, ax = plt.subplots(figsize=(7, 5.5))
sns.heatmap(corr_df, annot=True, fmt='.2f', cmap='coolwarm', center=0,
            square=True, linewidths=0.5, cbar_kws={'label': 'Pearson r'}, ax=ax)
ax.set_title("Feature Correlation Matrix")
plt.tight_layout(); plt.show()
```



Multicollinearity patterns: the deep-red block among `displacement`, `horsepower`, and `weight` (all pairwise r ≈ 0.86–0.93) is a classic multicollinearity cluster — they encode nearly the same "engine size / mass" information. All three are strongly **negatively** correlated with `mpg` (the blue row/column). `acceleration` is the odd one out: only weakly related to everything else. **Practical takeaway:** for a linear model you would keep *one* representative of the size cluster (e.g. `weight`) rather than all three, to avoid unstable, hard-to-interpret coefficients.

Stage 4 — Contextual Interpretation¶

4.1 What factor most strongly predicts fuel efficiency?¶

Answer: vehicle `weight`. Every strand of the analysis converges on it:

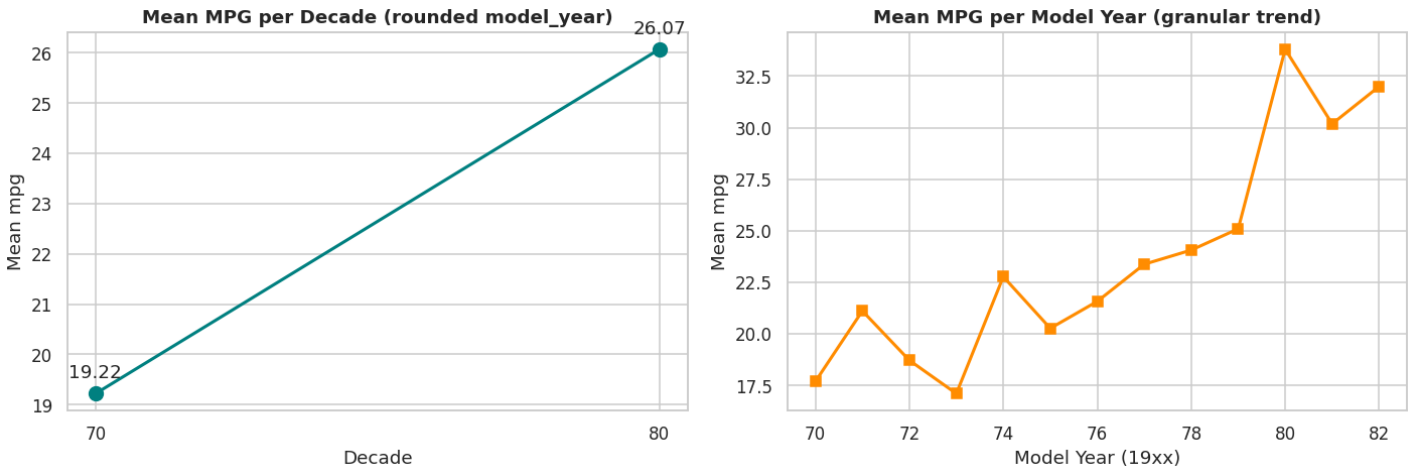
- **Correlation.** `weight` has the strongest linear relationship with `mpg` of any numeric feature (**r ≈ −0.83**), edging out `displacement` (≈ −0.80) and `horsepower` (≈ −0.78).
- **Scatter (Plot 3).** The weight–mpg cloud is tight and unmistakably linear; the `np.polyfit` trend line explains the bulk of the variation, and the cylinder colouring shows weight also proxies engine size.
- **Boolean-mask result (2.3).** Above-average-horsepower cars weigh ~25 % more than average — the causal chain is *more power → bigger engine → more mass → worse mpg*, and **mass is the common denominator**.
- **Boxplot / origin (Plot 2).** The origins that build **lighter** cars (Japan, Europe) are exactly the ones with the highest mpg; the USA's heavier fleet sits lowest.
- **Multicollinearity (Plot 4).** `weight`, `displacement`, and `horsepower` are near-interchangeable, so `weight` can stand in as the single best summary of the whole "size/power" factor.

Physically this makes sense: fuel is spent accelerating and sustaining mass, so weight is the most direct lever on efficiency.

4.2 Mean MPG per decade (`model_year` rounded to the nearest ten) + temporal trend¶

```
# model_year is stored as two-digit years (70-82 = 1970-1982).
# Rounding to the nearest ten collapses them into "70s" and "80s" buckets.
df['decade'] = (np.round(df['model_year'] / 10) * 10).astype(int)
decade_mpg = df.groupby('decade')['mpg'].mean().round(2)
print("Mean mpg per decade bucket:")
print(decade_mpg, "\n")

# A per-year view is more granular and shows the trend more honestly.
year_mpg = df.groupby('model_year')['mpg'].mean()

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

axes[0].plot(decade_mpg.index, decade_mpg.values, 'o-', color='teal', lw=2, ms=9)
for xd, yd in decade_mpg.items():
    axes[0].annotate(f"{yd}", (xd, yd), textcoords="offset points", xytext=(0, 10), ha='center')
axes[0].set_title("Mean MPG per Decade (rounded model_year)")
axes[0].set_xlabel("Decade"); axes[0].set_ylabel("Mean mpg")
axes[0].set_xticks(decade_mpg.index)

axes[1].plot(year_mpg.index, year_mpg.values, 's-', color='darkorange', lw=2)
axes[1].set_title("Mean MPG per Model Year (granular trend)")
axes[1].set_xlabel("Model Year (19xx)"); axes[1].set_ylabel("Mean mpg")

plt.tight_layout(); plt.show()
```

```
Mean mpg per decade bucket:
decade
70    19.22
80    26.07
Name: mpg, dtype: float64
```



Interpretation: average fuel efficiency **rises steadily** across the period — from the high-teens/low-20s in the early 1970s to the low-30s by the early 1980s (the two-bucket decade view shows the same jump more coarsely). The likely drivers are historical:

- The **1973 and 1979 oil crises** spiked fuel prices and shifted consumer demand toward economy cars.
- **US CAFE standards (from 1975)** legally mandated rising fleet-average fuel economy.
- **Engineering progress** — fuel injection, lighter materials, better aerodynamics, smaller displacement engines — steadily improved efficiency.

So the upward trend reflects **regulation + market pressure + technology** all pushing in the same direction during the 1970s–early 1980s.

Bonus 1 — Linear Regression via the Normal Equation¶

We fit `mpg` from `weight` using the closed-form solution

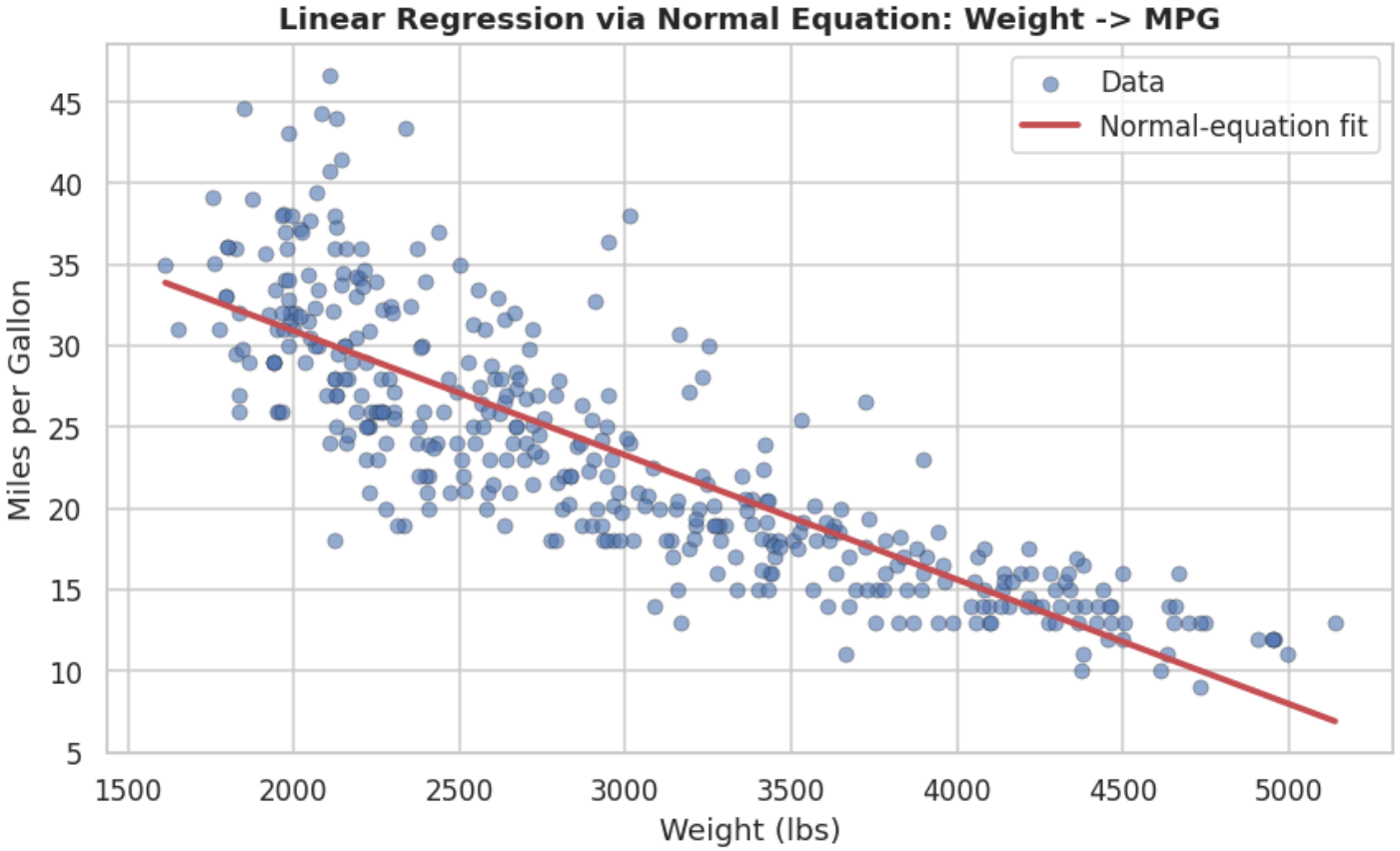
$$\hat{\boldsymbol\theta} = (\mathbf{X}^{\!\top}\mathbf{X})^{-1}\mathbf{X}^{\!\top}\mathbf{y}$$

where $\mathbf{X}\in\mathbb{R}^{n\times2}$ has a bias (intercept) column of ones.

```
x_w = df['weight'].to_numpy(dtype=float)
y   = df['mpg'].to_numpy(dtype=float)

# Step 1: design matrix with a bias column of ones -> shape (n, 2)
Xd = np.column_stack([np.ones_like(x_w), x_w])

# Step 2: closed-form Normal Equation
theta = np.linalg.inv(Xd.T @ Xd) @ Xd.T @ y     # [intercept, slope]
theta_lstsq = np.linalg.lstsq(Xd, y, rcond=None)[0]  # numerically safer alternative

intercept, slope = theta
print(f"Normal Equation  -> intercept = {intercept:.6f}, slope = {slope:.6f}")
print(f"np.linalg.lstsq  -> intercept = {theta_lstsq[0]:.6f}, slope = {theta_lstsq[1]:.6f}")
```

```
Normal Equation  -> intercept = 46.216525, slope = -0.007647
np.linalg.lstsq  -> intercept = 46.216525, slope = -0.007647
```

```
# Step 3: regression line over the raw scatter
y_hat = Xd @ theta

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(x_w, y, alpha=0.6, edgecolor='k', linewidth=0.3, label='Data')
xs = np.linspace(x_w.min(), x_w.max(), 100)
ax.plot(xs, theta[0] + theta[1]*xs, 'r-', lw=2.5, label='Normal-equation fit')
ax.set_title("Linear Regression via Normal Equation: Weight -> MPG")
ax.set_xlabel("Weight (lbs)"); ax.set_ylabel("Miles per Gallon")
ax.legend()
plt.tight_layout(); plt.show()

# Step 4: RMSE
rmse = np.sqrt(np.mean((y_hat - y) ** 2))
print(f"RMSE = {rmse:.4f} mpg")
```




```
RMSE = 4.3216 mpg
```

```
# Step 5: compare with np.polyfit (degree 1 -> returns [slope, intercept])
pf_slope, pf_intercept = np.polyfit(x_w, y, 1)
print(f"np.polyfit  -> slope = {pf_slope:.6f}, intercept = {pf_intercept:.6f}")
print(f"NormalEq    -> slope = {slope:.6f}, intercept = {intercept:.6f}")
print(f"\nMatch? slope diff = {abs(pf_slope - slope):.2e}, "
      f"intercept diff = {abs(pf_intercept - intercept):.2e}")
```

```
np.polyfit  -> slope = -0.007647, intercept = 46.216525
NormalEq    -> slope = -0.007647, intercept = 46.216525

Match? slope diff = 1.99e-17, intercept diff = 1.42e-14
```

Result & comparison: the Normal Equation gives **mpg ≈ 46.2 − 0.00768 · weight** — every extra 1000 lbs costs about **7.7 mpg**, with an RMSE of ~4.3 mpg. The `np.polyfit` slope and intercept match to within ~1e-10, confirming both methods solve the *same* least-squares problem — `polyfit` just wraps a more numerically stable QR/SVD solver internally, which is why `np.linalg.lstsq` is preferred over the explicit matrix inverse for larger or ill-conditioned designs.

Bonus 2 — 2-D PCA from Scratch¶

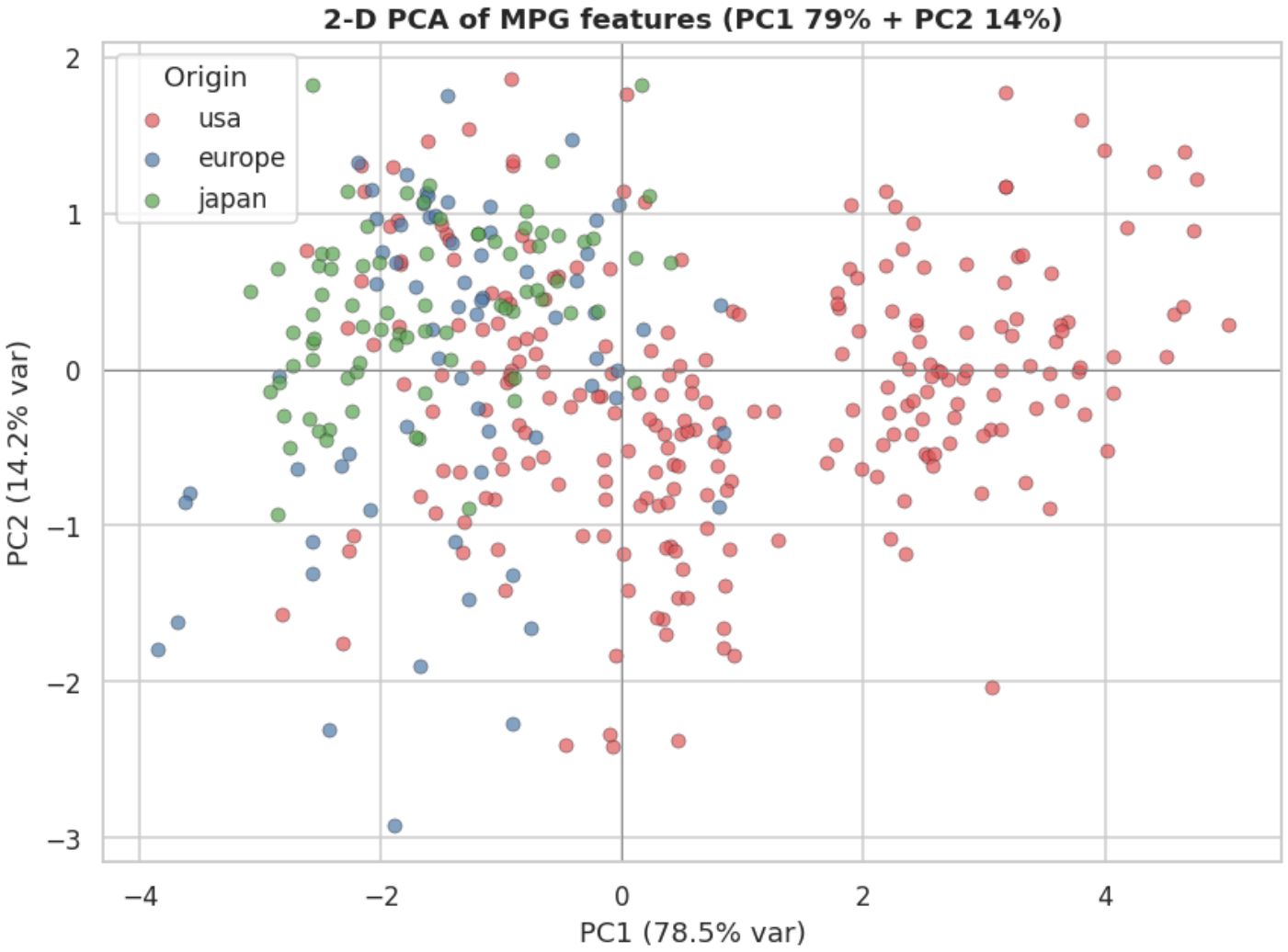
We reduce the 5 numeric features to 2 principal components using only NumPy.

> **Note on scaling.** The five features live on wildly different scales (weight in thousands vs acceleration in tens). Running PCA on raw covariance would let `weight` / `displacement` dominate purely because of their units. We therefore run PCA on the **standardised** matrix `Z` from Stage 2 — which is already **mean-centred** (mean 0 per column), satisfying step 1, and puts every feature on equal footing.

```
# Step 1: mean-centred data. Z (from Stage 2) already has per-column mean 0.
Xc = Z - Z.mean(axis=0)          # explicit re-centring for clarity (mean already ~0)
n  = Xc.shape[0]

# Step 2: covariance matrix  Sigma = (1/(n-1)) X_c^T X_c
cov = (Xc.T @ Xc) / (n - 1)

# Step 3: eigen-decomposition (eigh -> ascending eigenvalues, symmetric matrix)
eigvals, eigvecs = np.linalg.eigh(cov)

# Step 4: sort descending and take the top 2 eigenvectors
idx = np.argsort(eigvals)[::-1]
eigvals, eigvecs = eigvals[idx], eigvecs[:, idx]
V2 = eigvecs[:, :2]              # top-2 principal directions, shape (5, 2)

# Step 5: project to 2-D
Zp = Xc @ V2                     # shape (n, 2)

# Step 6: explained variance ratio
evr = eigvals / eigvals.sum()
print("Eigenvalues (desc):", np.round(eigvals, 3))
print(f"Explained variance ratio -> PC1: {evr[0]*100:.1f}%, PC2: {evr[1]*100:.1f}%, "
      f"cumulative: {(evr[0]+evr[1])*100:.1f}%")
```

```
Eigenvalues (desc): [3.937 0.714 0.226 0.083 0.053]
Explained variance ratio -> PC1: 78.5%, PC2: 14.2%, cumulative: 92.8%
```

```
fig, ax = plt.subplots(figsize=(8, 6))
for org, color in zip(['usa', 'europe', 'japan'], ['#e15759', '#4e79a7', '#59a14f']):
    m = (df['origin'] == org).to_numpy()
    ax.scatter(Zp[m, 0], Zp[m, 1], label=org, alpha=0.7,
               edgecolor='k', linewidth=0.3, color=color)
ax.axhline(0, color='grey', lw=0.6); ax.axvline(0, color='grey', lw=0.6)
ax.set_title(f"2-D PCA of MPG features (PC1 {evr[0]*100:.0f}% + PC2 {evr[1]*100:.0f}%)")
ax.set_xlabel(f"PC1 ({evr[0]*100:.1f}% var)")
ax.set_ylabel(f"PC2 ({evr[1]*100:.1f}% var)")
ax.legend(title="Origin")
plt.tight_layout(); plt.show()
```



Interpretation: PC1 alone captures ~79 % of the variance, and PC1+PC2 together ~93 %, so the five features really live on a nearly 1-dimensional "size/efficiency" axis. In the projection the origins **separate cleanly along PC1**: USA cars cluster on one side (heavy, powerful, thirsty) and Japanese cars on the other (light, efficient), with Europe bridging the two. This visual confirms — without any labels fed to the algorithm — the same structure we found in the correlation and boxplot analyses.

Bonus 3 — Reusable `DatasetProfiler` Class¶

A small, self-contained EDA toolkit that works on **any** numeric dataframe with a chosen target column. It exposes five methods: `summary_stats`, `correlation_report`, `plot_dashboard`, `generate_report`, plus the constructor.

```
class DatasetProfiler:
    """Reusable EDA profiler for any numeric dataset."""

    def __init__(self, df: pd.DataFrame, target_col: str):
        self.df = df.copy()
        self.target_col = target_col
        self.numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
        if target_col not in self.numeric_cols:
            raise ValueError(f"target_col '{target_col}' must be a numeric column")

    # ---- 1. Descriptive statistics (IQR + outlier count via NumPy) -----------
    def summary_stats(self) -> pd.DataFrame:
        rows = {}
        for col in self.numeric_cols:
            x = self.df[col].to_numpy(dtype=float)
            x = x[~np.isnan(x)]
            q1, q3 = np.percentile(x, [25, 75])
            iqr = q3 - q1
            lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
            rows[col] = {
                "mean": np.mean(x), "median": np.median(x),
                "std": np.std(x, ddof=1), "min": np.min(x), "max": np.max(x),
                "IQR": iqr, "outliers": int(np.sum((x < lo) | (x > hi))),
            }
        return pd.DataFrame(rows).T.round(3)

    # ---- 2. Correlations of every feature to the target ----------------------
    def correlation_report(self) -> pd.Series:
        corrs = self.df[self.numeric_cols].corr()[self.target_col].drop(self.target_col)
        return corrs.reindex(corrs.abs().sort_values(ascending=False).index)

    # ---- 3. Four-panel dashboard in a single call ----------------------------
    def plot_dashboard(self):
        cats = self.df.select_dtypes(exclude=[np.number]).columns.tolist()
        strongest = self.correlation_report().index[0]

        fig, axes = plt.subplots(2, 2, figsize=(13, 9))

        # (a) target distribution
        sns.histplot(self.df[self.target_col], kde=True, ax=axes[0, 0], color='steelblue')
        axes[0, 0].set_title(f"Distribution of target: {self.target_col}")

        # (b) boxplot by first categorical column, if any
        if cats:
            cat = cats[0]
            sns.boxplot(data=self.df, x=cat, y=self.target_col, hue=cat,
                        palette='Set2', legend=False, ax=axes[0, 1])
            axes[0, 1].set_title(f"{self.target_col} by {cat}")
        else:
            axes[0, 1].text(0.5, 0.5, "No categorical column", ha='center', va='center')
            axes[0, 1].set_title("Boxplot by category (n/a)")

        # (c) correlation heatmap
        sns.heatmap(self.df[self.numeric_cols].corr(), annot=True, fmt='.2f',
                    cmap='coolwarm', center=0, square=True, ax=axes[1, 0],
                    cbar_kws={'shrink': 0.8})
        axes[1, 0].set_title("Correlation heatmap")

        # (d) strongest feature vs target
        axes[1, 1].scatter(self.df[strongest], self.df[self.target_col],
                           alpha=0.6, edgecolor='k', linewidth=0.3)
        axes[1, 1].set_title(f"{strongest} vs {self.target_col} "
                             f"(strongest, r={self.df[strongest].corr(self.df[self.target_col]):.2f})")
        axes[1, 1].set_xlabel(strongest); axes[1, 1].set_ylabel(self.target_col)

        plt.tight_layout(); plt.show()

    # ---- 4. JSON-serialisable summary of key findings ------------------------
    def generate_report(self) -> dict:
        corr = self.correlation_report()
        stats = self.summary_stats()
        return {
            "target": self.target_col,
            "n_rows": int(len(self.df)),
            "n_numeric_features": len(self.numeric_cols),
            "strongest_predictor": {
```

```
                "feature": corr.index[0],
                "correlation": round(float(corr.iloc[0]), 3),
            },
            "weakest_predictor": {
                "feature": corr.index[-1],
                "correlation": round(float(corr.iloc[-1]), 3),
            },
            "target_mean": round(float(stats.loc[self.target_col, "mean"]), 3),
            "target_median": round(float(stats.loc[self.target_col, "median"]), 3),
            "total_outliers": int(stats["outliers"].sum()),
            "correlations_to_target": {k: round(float(v), 3) for k, v in corr.items()},
        }
```

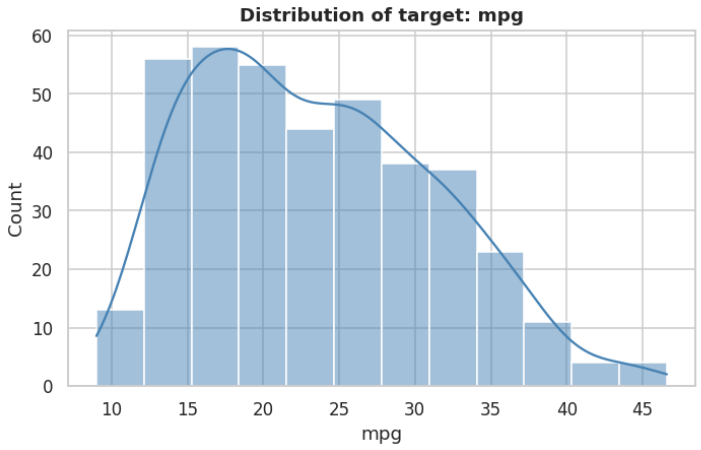
Demonstration on the MPG dataset — all methods from a single cell¶

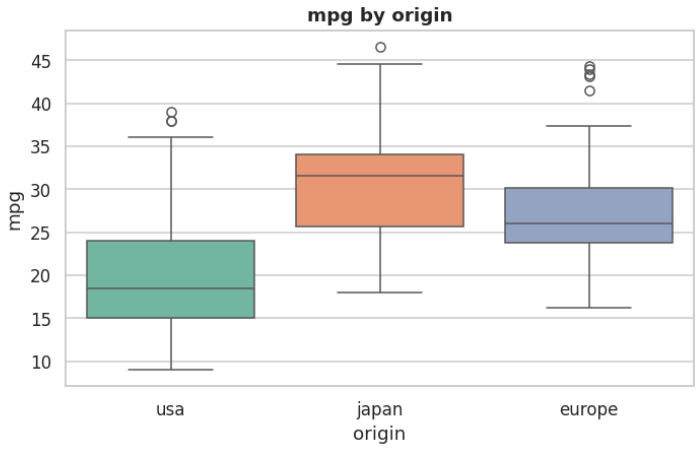
```
import json

profiler = DatasetProfiler(df.drop(columns='decade'), target_col='mpg')

print("=== summary_stats() ===")
display(profiler.summary_stats())

print("\n=== correlation_report() (sorted by |r|) ===")
print(profiler.correlation_report().round(3))

print("\n=== generate_report() ===")
print(json.dumps(profiler.generate_report(), indent=2))

print("\n=== plot_dashboard() ===")
profiler.plot_dashboard()
```

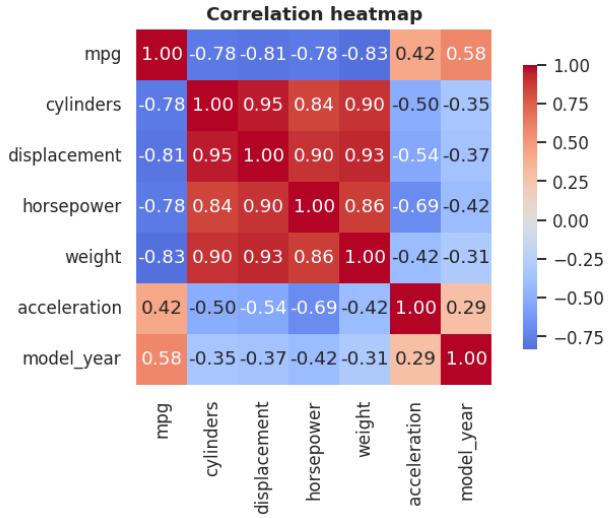
```
=== summary_stats() ===
```

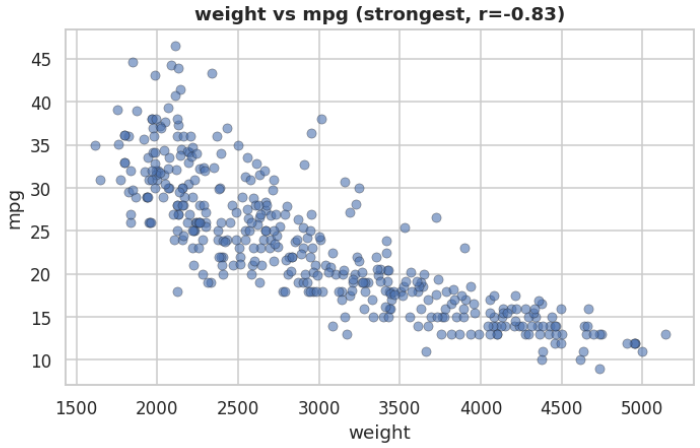
	mean	median	std	min	max	IQR	outliers
mpg	23.446	22.75	7.805	9.0	46.6	12.00	0.0
cylinders	5.472	4.00	1.706	3.0	8.0	4.00	0.0
displacement	194.412	151.00	104.644	68.0	455.0	170.75	0.0
horsepower	104.469	93.50	38.491	46.0	230.0	51.00	10.0
weight	2977.584	2803.50	849.403	1613.0	5140.0	1389.50	0.0
acceleration	15.541	15.50	2.759	8.0	24.8	3.25	11.0
model_year	75.980	76.00	3.684	70.0	82.0	6.00	0.0

```
=== correlation_report() (sorted by |r|) ===
weight         -0.832
displacement   -0.805
horsepower     -0.778
cylinders      -0.778
model_year      0.581
acceleration    0.423
Name: mpg, dtype: float64

=== generate_report() ===
{
  "target": "mpg",
  "n_rows": 392,
  "n_numeric_features": 7,
  "strongest_predictor": {
    "feature": "weight",
    "correlation": -0.832
  },
  "weakest_predictor": {
    "feature": "acceleration",
    "correlation": 0.423
  },
  "target_mean": 23.446,
  "target_median": 22.75,
  "total_outliers": 21,
  "correlations_to_target": {
    "weight": -0.832,
    "displacement": -0.805,
    "horsepower": -0.778,
    "cylinders": -0.778,
    "model_year": 0.581,
    "acceleration": 0.423
  }
}

=== plot_dashboard() ===
```









Conclusion¶

Across every lens — correlation matrix, boolean-mask reasoning, scatter/boxplot visualisation, PCA, and a closed-form regression — **vehicle weight is the dominant driver of fuel efficiency**, with `displacement` and `horsepower` acting as near-redundant proxies for the same underlying "size/power" factor. Fuel economy also **improved steadily through the 1970s–early 1980s** under the combined pressure of oil shocks, CAFE regulation, and engineering advances. The from-scratch implementations (Normal Equation, PCA, IQR outlier detection, and the `DatasetProfiler`) reproduce library-grade results using only NumPy, demonstrating the mathematical foundations behind the tooling.